

Esercizio A1 – Complessità – Punti ≥ 8

Dal codice, è possibile vedere che la matrice di input è quadrata. Consideriamo, per semplicità, il caso in cui n è una potenza di 2. Il risultato è valido anche per dimensioni che non rispettano tale vincolo.

Il codice calcola la somma dei valori lungo i bordi della matrice, in tempo $O(n)$, e poi chiama ricorsivamente se stessa su due porzioni di dimensione $n/2$, che corrispondono al secondo e quarto quadrante se interpretiamo la matrice come un piano cartesiano.

La funzione di complessità è quindi:

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 2T(n/2) + n & n > 1 \end{cases}$$

Applicando il Master Theorem, otteniamo $\alpha = \log_2 2 = 1$ e $\beta = 1$. Poichè $\beta = \alpha$, per il Master Theorem $T(n) = \Theta(n \log n)$.

Esercizio A2 – Somma su albero – Punti ≥ 10

Il problema può essere risolto con una semplice visita in profondità. Assumiamo che ∞ sia un valore speciale che non compare in nessun nodo. Il costo computazionale è ovviamente quello di una visita di un albero, cioè $O(n)$.

boolean isSumTree(TREE T)
int $sum = \text{isRec}(T)$
return $sum \neq \infty$ and $T.value == sum$
int isRec(TREE T)
if $T == \text{nil}$ then
return 0
else if $T.left() == \text{nil}$ and $T.right() == \text{nil}$ then
return $T.value$
int $sum_L = \text{isRec}(T.left())$
int $sum_R = \text{isRec}(T.right())$
if $sum_L \neq \infty$ and $sum_R \neq \infty$ and $T.value == sum_L + sum_R$ then
return $T.value \cdot 2$
else
return ∞

Esercizio A3 – Tree fire – Punti ≥ 12

L'algoritmo proposto effettua una visita in profondità dell'albero come se fosse un grafo, a partire dal nodo t . Invece di memorizzare informazioni sulla visita sotto forma di valori visited (che dovrebbero essere inseriti all'interno dell'albero oppure in qualche forma di tabella hash), l'algoritmo semplicemente evitare di tornare sui propri passi, sfruttando il fatto che non esistono cicli. Viene quindi passato il riferimento $prev$ al nodo precedente, che serve ad evitare di tornare indietro.

Il costo computazionale è quello di una visita in profondità, cioè $O(n)$.

int fire(TREE t)
return fireRec(t, nil)
int fireRec(TREE t , TREE $prev$)
int $max = 0$
if $t.left \neq \text{nil}$ and $t.left \neq prev$ then
$max = \max(max, 1 + \text{fireRec}(t.left, t))$
if $t.right \neq \text{nil}$ and $t.right \neq prev$ then
$max = \max(max, 1 + \text{fireRec}(t.right, t))$
if $t.parent \neq \text{nil}$ and $t.parent \neq prev$ then
$max = \max(max, 1 + \text{fireRec}(t.parent, t))$
return max

Esercizio B1 – Misciotto 1 – Punti ≥ 8

Dovendo stampare un insieme combinatorio di stringhe, utilizziamo un algoritmo di backtracking. La funzione `printMixing()` è un wrapper che richiama `printRec`, dopo aver creato un vettore T contenente spazio per $n_1 + n_2$ caratteri.

La funzione ricorsiva lavora dal fondo, per passare meno parametri. Quando entrambi n_1 e n_2 sono a zero, la stringa viene stampata; altrimenti, la funzione viene chiamata ricorsivamente, prima aggiungendo un carattere della prima stringa (se possibile), poi della seconda (se possibile).